MAGYAR AGRÁR- ÉS ÉLETTUDOMÁNYI EGYETEM

INTÉZMÉNYI TUDOMÁNYOS DIÁKKÖRI KONFERENCIA

2026. november 18.

TUTORCONNECT – KORREPETÁLÓ-KÖZVETÍTŐ WEBALKALMAZÁS

TutorConnect – A web application connecting students and tutors

Készítette:

Porkoláb Mátyás Attila **Stickel Zsombor** **Marián Máté**

Szoftverfejlesztő és -tesztelő technikus

Kaposvár

Témavezető: [Témavezető neve], oktató, KIAKK, Kaposvár

Lehetséges OTDK szekció: **Informatika**

Kaposvár

2026.

Tartalomjegyzék

1. Bevezetés....3

2. Célkitűzés....4

2.1 Funkcionális követelmények....4

2.2 Nem funkcionális követelmények....4

3. Szakirodalmi áttekintés....6

3.1 Futtatókörnyezet és szerveroldali keretrendszer....6

3.2 Adatbázis-kezelés....6

3.3 Szabványok és biztonsági ajánlások....7

3.4 Kliensoldali technológiák....7

3.5 Fejlesztői és projektmenedzsment eszközök....7

3.6 Mesterséges intelligencia alapú eszközök....7

3.7 Adatok forrása....8

4. Eredmények....9

4.1 A rendszer fő funkciói....9

4.2 Felhasználói szerepkörök....9

4.3 Rendszerterv és architektúra....10

4.4 Adatbázis....12

4.5 Felhasználói felület....14

4.6 Szerveroldali megvalósítás (backend)....15

4.7 Kliensoldali megvalósítás (frontend)....17

4.8 Telepítés és üzemeltetés....18

4.9 Tesztelés....18

4.10 Forráskód és projektstruktúra....20

4.11 Csapatmunka....21

5. Következtetések, javaslatok, összegzés....22

5.1 Tapasztalatok....22

5.2 Továbbfejlesztési javaslatok....22

5.3 Összegzés....23

6. Irodalomjegyzék....24

1. Bevezetés

Sok diák szembesül azzal, hogy egy-egy tantárgyból lemarad, és külső segítségre, korrepetálásra volna szüksége. Ilyenkor rendszerint ismerősöktől kér ajánlást, vagy hirdetési oldalakon és közösségi csoportokban keres, ahol a korrepetálók ajánlatai szétszórtan, nehezen összehasonlítható formában jelennek meg. A korrepetálók oldaláról nézve ugyanez a probléma fordítva jelentkezik: a hirdetéseiket alig veszik észre, és nincs olyan felület, ahol a tantárgyaikat, óradíjukat és a korábbi diákok visszajelzéseit egy helyen bemutathatnák.

A jelenlegi helyzet főbb hiányosságai a következők:

- a diákok nem tudják, hol keressenek megbízható korrepetálót;
- nincs olyan közös felület, ahol a tantárgyak és az óradíjak összehasonlíthatók;
- nem látszik, hogy a korábbi diákok mit gondoltak egy-egy oktatóról;
- az időpont-egyeztetés hosszadalmas, sokszor több üzenetváltást igényel.

Pályamunkánkban erre a problémára készítettünk megoldást. A **TutorConnect** egy webalkalmazás, amelyben a diákok és a korrepetálók egyetlen felületen találnak egymásra: a diák kiválasztja a tantárgyat, összehasonlítja az oktatók óradíját és értékelését, majd üzenetet ír vagy időpontot foglal. Az óra után értékelést adhat, amely a későbbi diákok döntését segíti.

A rendszer célcsoportja két fő felhasználói kört foglal magában. A **diákok** között általános iskolások, középiskolások és egyetemi hallgatók egyaránt megtalálhatók, akik egy vagy több tantárgyból keresnek segítséget. Az **oktatók** (korrepetálók) lehetnek segítőkész diákok, tanárok, egyetemi hallgatók vagy oktatók, akik órákat adnának. Rajtuk kívül a rendszer ismeri a **vendég** (be nem jelentkezett látogató) és az **adminisztrátor** szerepkört is.

A projektet egy háromfős csapat (Porkoláb Mátyás Attila, Stickel Zsombor és Marián Máté) készítette 2026 szeptembere és októbere között. A dolgozat a következő fejezetekben bemutatja a célkitűzéseket, a felhasznált forrásokat és technológiákat, az elkészült rendszert, végül összegzi a tapasztalatokat és a továbbfejlesztési lehetőségeket.

2. Célkitűzés

Célunk egy olyan webes platform létrehozása volt, amely lerövidíti a diák és a korrepetáló közötti kapcsolatfelvétel útját: a keresést, az egyeztetést, a foglalást és az értékelést egyetlen rendszerbe szervezi. Konkrétan a következőket kívántuk elérni:

- a diák gyorsan találjon oktatót abból a tantárgyból, amelyből gyengébb;
- a korrepetáló egy helyen jelenhessen meg, és a diákok észrevegyék;
- a kapcsolatfelvétel, az egyeztetés és a foglalás egy oldalon megoldható legyen;
- az értékelések alapján a diák magabiztosan választhasson oktatót.

2.1 Funkcionális követelmények

A célokból az alábbi funkcionális követelményeket vezettük le:

1. táblázat: Funkcionális követelmények

Követelmény	**Tartalma**
Regisztráció, bejelentkezés	Minden mező kötelező, a két jelszónak egyeznie kell; a szerepkör (diák vagy korrepetáló) a regisztrációkor választható. Belépés után a munkamenet 7 napig érvényes.
Tantárgyak szűrése	42 tantárgy 7 kategóriában (pl. reál, humán, nyelvek), kategóriánként szűrhetően.
Oktatók keresése	Az oktatók listája tantárgy és óradíj szerint kereshető és szűrhető.
Ár alapú szűrés	Az óradíj tantárgyanként van megadva, és szűrési feltételként használható.
Chat	Valós idejű beszélgetés diák és oktató között, kép- és dokumentumküldéssel; az üzenet szerkeszthető és visszavonható.
Foglalás	Szabad időpont kiválasztása, a kérés elfogadása vagy elutasítása, lemondás.
Értékelés	Az óra után 1–5 csillagos értékelés és rövid szöveges vélemény.
GYIK	Gyakran ismételt kérdések külön oldalon.

2.2 Nem funkcionális követelmények

- **Biztonságos jelszókezelés:** a jelszót soha nem tároljuk nyílt szövegként, csak az Argon2id algoritmussal képzett hash-értéke kerül az adatbázisba.
- **Süti alapú munkamenet:** a böngésző csak egy munkamenet-azonosítót kap, az adatok a szerveren maradnak.
- **Jogosultságkezelés és adatvédelem:** a szerver forráskódja nem érhető el a böngészőből, a feltöltött fájlokat csak a beszélgetés két résztvevője nyithatja meg.

- **Reszponzivitás és felhasználói élmény:** az oldal telefonon és asztali monitoron is használható; világos és sötét téma, animációk, statisztikák a főoldalon.

3. Szakirodalmi áttekintés

A fejlesztés során elsősorban a felhasznált szoftverek és programcsomagok hivatalos dokumentációjára, a webes kommunikációt és a jelszókezelést leíró szabványokra, valamint biztonsági ajánlásokra támaszkodtunk. Emellett mesterséges intelligencián alapuló asszisztenst is használtunk. Az alábbiakban forráscsoportonként bemutatjuk, melyik forrást mire használtuk.

3.1 Futtatókörnyezet és szerveroldali keretrendszer

A szerveroldali JavaScript futtatási környezet kialakításához a Node.js-t használtuk (OpenJS Foundation, 2026a). Választásunk fő oka az volt, hogy így a kliens és a szerver ugyanazon a nyelven, JavaScriptben készülhetett. A csomagok telepítését és a szkriptek (`npm run dev`, `npm test`) futtatását az npm csomagkezelő végzi. A Node.js beépített tesztfuttatóját (`node:test`) az automatizált tesztek megírásához használtuk, ennek működését a hivatalos dokumentációból sajátítottuk el.

A webszervert és a REST API-t az Express 5 keretrendszerre építettük (OpenJS Foundation, 2026b). Az Express dokumentációja alapján alakítottuk ki az útvonalkezelést, a statikus fájlok kiszolgálását és a köztes rétegeket (middleware). A REST architektúra elveit – az erőforrás-alapú címzést és az állapotmentes kéréseket – Fielding (2000) doktori értekezése fogalmazta meg; API-nk végpontjait ennek szellemében terveztük.

A fontosabb npm-csomagok és azok szerepe a projektben:

2. táblázat: A felhasznált npm-csomagok

Csomag (verzió)	Felhasználás	Hivatkozás
express (5.2.1)	Webszerver és REST API	OpenJS Foundation (2026b)
express-session (1.19.0)	Süti alapú munkamenet-kezelés	Express.js (2026a)
mysql2 (3.24.5)	Kapcsolat a MySQL/MariaDB adatbázissal	Sidorov (2026)
argon2 (0.45.1)	Jelszó hash-elés Argon2id-vel	Althoff (2026)
multer (2.4.0)	Fájlfeltöltés a chatben (max. 10 MB)	Express.js (2026b)
ws (8.22.0)	WebSocket alapú valós idejű kommunikáció	Pinca (2026)

3.2 Adatbázis-kezelés

Az adatbázis-szervert a XAMPP csomag biztosította (Apache Friends, 2026), amely MariaDB adatbázis-kezelőt tartalmaz. Az SQL-utasítások, az idegen kulcsok, a kaszkádolt

törlés (`ON DELETE CASCADE`) és az utf8mb4 karakterkészlet beállításánál a MariaDB hivatalos dokumentációját használtuk (MariaDB Foundation, 2026). Az adatbázis grafikus kezelését, az importálást és az exportálást (dump) a phpMyAdmin felületén végeztük (phpMyAdmin Project, 2026).

3.3 Szabványok és biztonsági ajánlások

A valós idejű chat a WebSocket protokollra épül, amelyet az RFC 6455 szabvány ír le (Fette & Melnikov, 2011). Ebből értettük meg, hogyan jön létre a tartós, kétirányú kapcsolat a böngésző és a szerver között, és miért alkalmasabb ez az azonnali értesítésekre, mint az ismételt HTTP-lekérdezés.

A jelszavak tárolására az Argon2id algoritmust választottuk. Az algoritmust az RFC 9106 szabványosította (Biryukov et al., 2021), és az OWASP jelszótárolási ajánlása is elsődlegesen ezt javasolja (OWASP Foundation, 2026). Ugyanez az ajánlás erősítette meg bennünket abban, hogy a jelszót soha ne tároljuk visszafejthető formában. A feltöltött fájlok kezelésénél (véletlen fájlnév, kiterjesztés-ellenőrzés, hozzáférés-szabályozás) szintén az OWASP útmutatásait vettük figyelembe.

3.4 Kliensoldali technológiák

A felhasználói felület keretrendszer nélkül, tiszta HTML, CSS és JavaScript nyelven készült. A nyelvi elemek, a `fetch` API, a WebSocket kliensoldali használata, valamint a reszponzív megjelenéshez szükséges `@media` szabályok tekintetében az MDN Web Docs referenciáját használtuk (Mozilla, 2026).

3.5 Fejlesztői és projektmenedzsment eszközök

A forráskódot a Visual Studio Code szerkesztőben írtuk (Microsoft, 2026a). A projekthez tartozó asztali alkalmazás vázát C# nyelven, Windows Forms technológiával a Visual Studio fejlesztőkörnyezetben és a .NET 10 platformon kezdtük el (Microsoft, 2026b, 2026c). A verziókezelést Gittel végeztük (Git Project, 2026), a közös tárolót a GitHub szolgáltatta (GitHub, 2026). Ez tette lehetővé, hogy a három csapattag párhuzamosan dolgozzon, és a változásokat a fő ágban egyesítse.

3.6 Mesterséges intelligencia alapú eszközök

A fejlesztés során az Anthropic (2026) Claude nevű nagy nyelvi modelljét használtuk tanácsadásra, kódellenőrzésre és egyes kódrészletek megírására. A modell által javasolt megoldásokat minden esetben átnéztük, teszteltük, és szükség esetén módosítottuk. A

projektdokumentáció végleges szövegezése szintén mesterséges intelligencia segítségével készült el, a csapat által írt kezdeti szöveg alapján (Anthropic, 2026).

3.7 Adatok forrása

Az alkalmazás nem használ külső adatforrást. A 42 tantárgyat és a 7 tantárgykategóriát a csapat saját maga állította össze a hazai köznevelési és felsőoktatási tantárgyak alapján. A tesztelésre használt minta-felhasználók, oktatók, foglalások és üzenetek szintén saját készítésű, fiktív adatok, amelyeket az `sql/zsamo.sql` adatbázis-dump tartalmaz.

4. Eredmények

Ebben a fejezetben bemutatjuk az elkészült rendszert: az architektúrát, az adatbázist, a felhasználói felületet, a szerver- és kliensoldali megvalósítást, a telepítés menetét, a tesztelés eredményeit, valamint a csapatmunka szervezését.

4.1 A rendszer fő funkciói

A felhasználó útja öt lépésből áll: **regisztráció** (fiók létrehozása és szerepkör választása), **keresés** (tantárgy és óradíj szerinti szűrés), **chat** (egyeztetés az oktatóval), **foglalás** (időpont kérése és elfogadása), végül **értékelés** (csillagok és vélemény az óra után).

3. táblázat: A rendszer fő funkciói

Funkció	Leírás
Regisztráció	Fiók létrehozása; itt kell kiválasztani, hogy a felhasználó diák vagy korrepetáló lesz.
Bejelentkezés	Belépés e-mail-címmel és jelszóval; a munkamenet 7 napig megmarad.
Tantárgyszűrés	42 tantárgy 7 kategóriában (pl. reál, humán, nyelvek).
Oktatókeresés	Az oktatók listája tantárgy és óradíj szerint szűrhető.
Chat	Valós idejű beszélgetés; képek és dokumentumok is küldhetők.
Foglalás	A diák időpontot kér, az oktató elfogadja vagy elutasítja; lemondani is lehet.
Értékelés	A lezajlott óra után 1–5 csillag és rövid vélemény adható.
Profil	Bemutatkozás, tantárgyak és óradíj szerkesztése, sötét mód.

4.2 Felhasználói szerepkörök

A rendszer négy szerepkört különböztet meg. A szerepkört az adatbázis `users` táblájának `role` oszlopa tárolja.

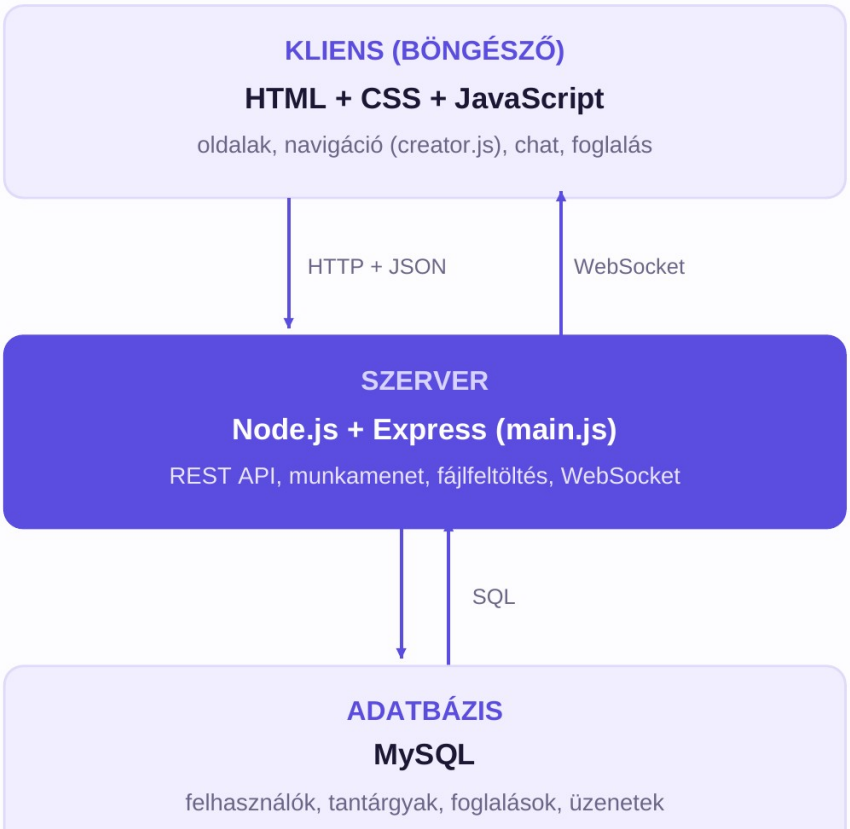
- **Vendég** (nincs bejelentkezve): tantárgyakat és oktatókat böngészhet, olvashatja a GYIK-et és az értékeléseket, de foglalni és csevegni nem tud.
- **Diák:** csevegni tud az oktatókkal, időpontot foglalhat, látja a közelgő foglalásait és lemondhatja őket; az óra után értékelheti az oktatót, és szerkesztheti a profilját (iskolatípus, osztály, tantárgyak, amelyekből segítséget kér).
- **Korrepetáló:** csevegni tud a diákokkal, megnézheti a beszélgetőpartner diák profilját, megadja szabad időpontjait, elfogadja vagy elutasítja a foglalási kérelmeket, szerkeszti a bemutatkozását, tantárgyait és óradíját.
- **Admin:** minden oldalt lát. Admin fiókot regisztrációval nem lehet létrehozni, csak közvetlenül az adatbázisban.

4. táblázat: Jogosultsági mátrix

Funkció	Vendég	Diák	Korrep.	Admin
Tantárgyak és oktatók böngészése	✓	✓	✓	✓
GYIK és értékelések olvasása	✓	✓	✓	✓
Csevegés	–	✓	✓	✓
Időpontfoglalás	–	✓	–	✓
Foglalás elfogadása / elutasítása	–	–	✓	✓
Szabad időpontok megadása	–	–	✓	✓
Oktató értékelése	–	✓	–	✓
Profil szerkesztése	–	✓	✓	✓

4.3 Rendszerterv és architektúra

A TutorConnect klasszikus háromrétegű architektúrára épül. A böngésző kéri az adatot, a szerver feldolgozza, az adatbázis tárolja. A böngésző soha nem éri el közvetlenül az adatbázist, minden kérés a szerveren (`main.js`) halad át. A szerver a `http://localhost:3000` címen érhető el.



1. ábra: A rendszer háromrétegű architektúrája

5. táblázat: Az architektúra rétegei

Réteg	Technológia	Helye a projektben
Kliens	HTML, CSS, JavaScript	vsc/html, vsc/css, vsc/js (a böngészőben futó fájlok)
Szerver	Node.js + Express	vsc/js/main.js és vsc/js/auth.js
Adatbázis	MySQL / MariaDB	sql/zsamo.sql (szerkezet és mintaadatok)

A rendszer komponensei

6. táblázat: A rendszer fő komponensei

Fájl	Feladat
main.js	A szerver: kiszolgálja az oldalakat, kezeli a bejelentkezést, a chatet és a foglalásokat, és egyedül ez éri el az adatbázist (kb. 3850 sor).
auth.js	Jelszó hash-elése és ellenőrzése Argon2id-vel.
creator.js	Minden oldalon felépíti a navigációs sávot, a profilmenüt és a témaváltót.
home.js	A főoldal: statisztikák és diavetítés.
index.js	A tantárgyak oldal logikája.
oktatok.js	Az oktatók listája, szűrése és az értékelések megjelenítése.
dropdown.js, selectmenu.js	Egyedi legördülő menük a tantárgyak és az oktatók oldalán.
booking.js	Az időpontfoglalás ablaka és folyamata.
foglalas.js	A foglalás oldal: szabad időpontok, saját foglalások.
foglalasutan.js	A foglalások utáni oldal: állapot, lemondás, értékelés.
chat.js	A chat oldal: WebSocket-kapcsolat, csatolmányok, képnézegető.

A `main.js` és az `auth.js` a szerveren fut, a többi fájlt a szerver kiszolgálja, és azok a böngészőben futnak.

Kommunikáció a komponensek között

- **REST API:** a böngésző `fetch` kéréseket küld a szervernek, amely JSON formátumban válaszol.
- **WebSocket:** új üzenet vagy foglalásváltozás esetén a szerver azonnal értesíti a másik felet.
- **Süti (munkamenet):** minden kérés magával viszi a `connect.sid` sütit, így a szerver tudja, ki küldte a kérést.
- **Adatbázis:** csak a `main.js` éri el a MySQL-t, a böngésző soha.

Példaként egy üzenet elküldése így zajlik: (1) a `chat.js` POST kérést küld a szervernek; (2) a `main.js` ellenőrzi, hogy a felhasználó be van-e jelentkezve és tagja-e a beszélgetésnek, majd az üzenetet az adatbázisba írja; (3) a szerver WebSocketen azonnal továbbítja az új üzenetet a másik félnek.

REST API

A kliens és a szerver közötti kommunikáció REST elveket követ (Fielding, 2000): a böngésző HTTP-kéréseket küld, az adatok JSON formában közlekednek. Hiba esetén a válasz egységes formájú, például: `{ "error": "Nem vagy bejelentkezve." }`.

7. táblázat: A használt HTTP-metódusok

Metódus	Szerepe	Példa
GET	Adat lekérése	GET /api/tutors
POST	Új adat létrehozása	POST /api/bookings
PUT	Meglévő adat módosítása	PUT /api/account
PATCH	Részleges módosítás	PATCH /api/messages/:id
DELETE	Törlés	DELETE /api/availability/:id

Alkalmazott technológiák és indoklásuk

8. táblázat: Technológiaválasztás

Terület	Technológia	Indoklás
Backend	Node.js + Express 5	Egy nyelven készül a szerver és a kliens.
Adatbázis	MySQL / MariaDB (XAMPP)	Relációs táblák és kapcsolatok, phpMyAdminban könnyen kezelhető.
Frontend	HTML, CSS, JavaScript	Nincs keretrendszer, minden oldal külön fájl.
Valós idő	ws (WebSocket)	A chat és a foglalások változásai azonnal megjelennek.
Munkamenet	express-session	Süti alapú belépés, 7 napig.
Fájlfeltöltés	multer	Képek és dokumentumok a chatben, legfeljebb 10 MB.
Jelszó	Argon2id	Biztonságos, szabványos hash-elés (Biryukov et al., 2021).
Verziókezelés	Git + GitHub	Közös munka, a változások követése.

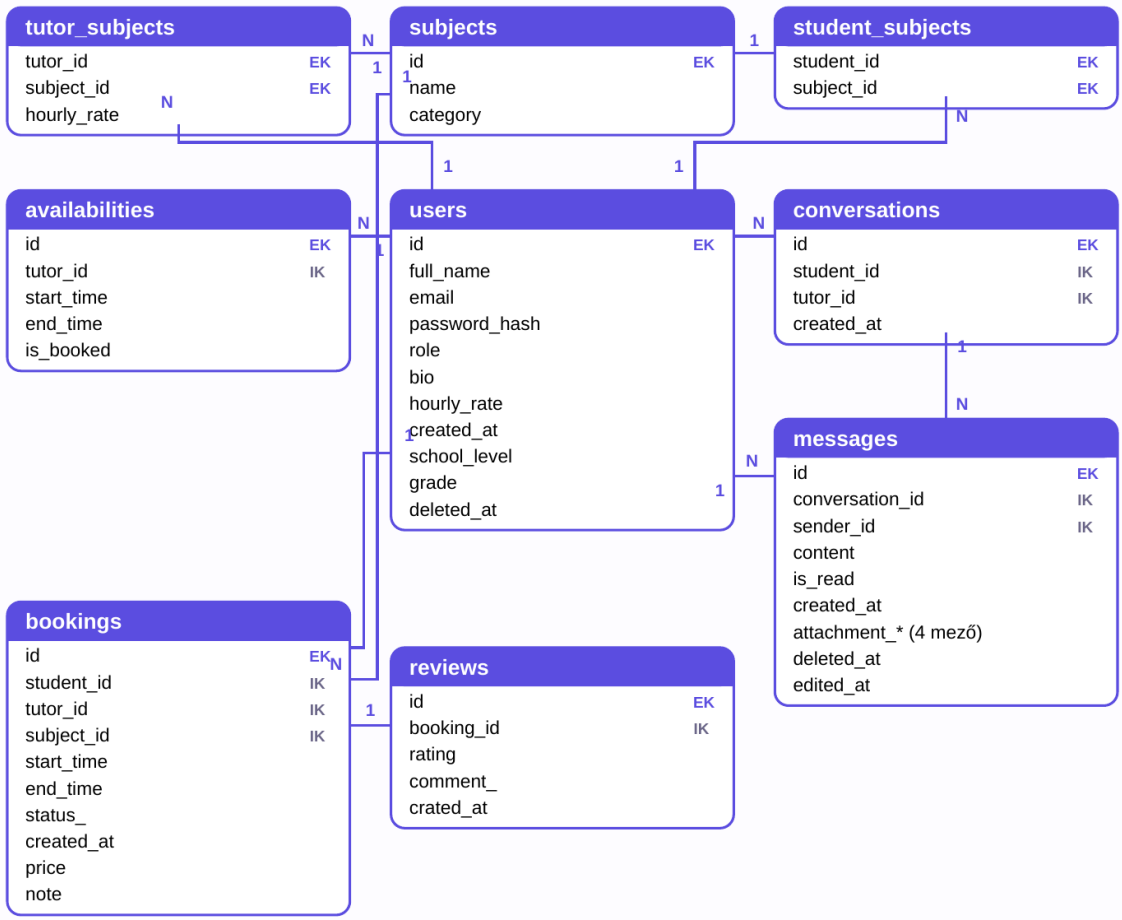
A projekt része egy C# Windows Forms asztali alkalmazás váza is (`Desktop` mappa), amely egyelőre csak a navigációs sávot (Profil, Beállítások) tartalmazza.

4.4 Adatbázis

Az adatokat egy `zsamo` nevű MySQL/MariaDB adatbázisban tároljuk, utf8mb4 karakterkészlettel és magyar rendezéssel (`utf8mb4_hungarian_ci`). Az adatbázis 9 táblából áll, amelyek között 13 kapcsolat van. Fontosabb tervezési döntéseink:

- egy beszélgetés mindig egy diák és egy oktató között jön létre, ugyanaz a páros csak egyszer szerepelhet;
- egy foglaláshoz legfeljebb egy értékelés tartozhat;

- az oktatók óradíját tantárgyanként tároljuk;
- a törölt fiókot nem töröljük fizikailag, hanem anonimizáljuk (név, e-mail és jelszó törlődik, a `deleted_at` mező kitöltődik), így a régi foglalások és üzenetek nem sérülnek;
- a foglalás állapotai: PENDING (várakozik), CONFIRMED (elfogadva), REJECTED (elutasítva), COMPLETED (megtörtént), CANCELLED (lemondva).



2. ábra: Az adatbázis ER-diagramja (EK: elsődleges kulcs, IK: idegen kulcs)

9. táblázat: Az adatbázis táblái

Tábla	Tartalma
users	A felhasználók adatai: azonosító, név, e-mail-cím, jelszó hash, szerepkör (diák, korrepetáló, admin), bemutatkozás, iskolatípus, osztály, törlés időpontja.
subjects	A tantárgyak és kategóriáik.
tutor_subjects	Összekötő tábla: melyik oktató melyik tantárgyat tanítja, és milyen óradíjjal.
student_subjects	Összekötő tábla: a diák mely tantárgyakból kér segítséget.
availabilities	Az oktatók szabad időpontjai (kezdés, befejezés, foglalt-e már).
bookings	A foglalások: diák, oktató, tantárgy, időpont, ár, megjegyzés és állapot.

Tábla	Tartalma
reviews	Az értékelések: melyik foglaláshoz tartoznak, csillagszám, szöveges vélemény.
conversations	Kik között jött létre beszélgetés.
messages	Az üzenetek: küldő, beszélgetés, tartalom, olvasottság, csatolmány adatai, szerkesztés és törlés ideje.

10. táblázat: A táblák közötti kapcsolatok

Kapcsolat	Típus	Mi köti össze
users – availabilities	1:N	Az oktató szabad időpontjai (tutor_id).
users – bookings	1:N	A diák és az oktató is (student_id, tutor_id).
subjects – bookings	1:N	Melyik tantárgyból szól a foglalás.
bookings – reviews	1:1	Egy foglaláshoz egy értékelés.
users – conversations	1:N	A diák és az oktató beszélgetése.
conversations – messages	1:N	Egy beszélgetés üzenetei.
users – messages	1:N	Az üzenet küldője (sender_id).
users – subjects	N:M	Összekötő táblák: tutor_subjects és student_subjects.

A `conversations`, `messages` és `tutor_subjects` táblák kapcsolatai kaszkádolt törlést alkalmaznak (`ON DELETE CASCADE`), a többi kapcsolat csak hivatkozási kényszer. Példa egy idegen kulcsra:

```
ALTER TABLE `bookings`
  ADD CONSTRAINT `bookings_ibfk_1` FOREIGN KEY (`student_id`)
  REFERENCES `users` (`id`);
```

Az adatbázis-dump (`sql/zsamo.sql`) tartalmazza minden tábla szerkezetét, a 42 tantárgyat, valamint néhány minta-felhasználót, oktatót, foglalást és üzenetet. A szerver induláskor szükség esetén automatikusan kiegészíti a sémát (például az üzenetek csatolmány-oszlopaival és az e-mail mező hosszának bővítésével).

4.5 Felhasználói felület

11. táblázat: Az alkalmazás oldalai

Oldal	Útvonal	Ki látja	Tartalma
Főoldal	/index	mindenki	Nyitóoldal statisztikákkal, a regisztrációhoz vagy belépéshez vezet.
Tantárgyak	/subs	mindenki	A tantárgyak böngészése kategóriák szerint.
Oktatók	/oktatok	mindenki	Az oktatók listája óradíj szerinti szűréssel és értékelésekkel.
Chat	/chat	bejelentkezve	Beszélgetés a korrepetálóval, fájlküldéssel.

Oldal	Útvonal	Ki látja	Tartalma
Foglalás	/foglalas, /foglalasutan	bejelentkezve	Szabad időpontok és a saját foglalások állapota.
GYIK	/gyik	mindenki	Válaszok a gyakran ismételt kérdésekre.
Belépés, regisztráció	/login, /register	mindenki	Belépés meglévő fiókba vagy új fiók létrehozása.

Navigáció

A navigációs sávot a `creator.js` építi fel minden oldalon az `/api/me` végpont válasza alapján, így a menü a szerepkörtől függően eltér:

- **Vendég:** Főoldal, Tantárgyak, GYIK, Belépés, Regisztráció;
- **Diák:** Főoldal, Chat, Tantárgyak, Oktatók, Foglalás, Profil;
- **Korrepetáló:** Főoldal, Chat, Tantárgyak, Foglalás, Profil.

Bejelentkezés után megjelenik a profilmenü a felhasználó nevének kezdőbetűivel (fiókbeállítások, kijelentkezés), valamint minden oldalon elérhető a világos és sötét téma közötti váltó gomb.

Reszponzív működés

A reszponzivitást a CSS kezeli `@media` szabályokkal, amelyek többek között 600, 768, 900 és 1200 pixeles szélességnél váltanak. Kisebb képernyőn az elemek egymás alá kerülnek, és a menü is átalakul. Az oldal megjelenését telefonon és közepes méretű monitoron teszteltük; régebbi, 4:3 arányú monitorokon és okosórákon nem próbáltuk ki.

4.6 Szerveroldali megvalósítás (backend)

A szerver indulásakor a `main.js` sorrendben beállítja az Express-t (JSON és űrlapadatok feldolgozása), a munkamenetet (`connect.sid` süti, httpOnly, 7 napos élettartam, a titkos kulcs a `SESSION_SECRET` környezeti változóból), a statikus fájlok kiszolgálását (csak fehérlistás kliensfájlok, a `main.js` és az `auth.js` nem érhető el), az adatbázis-kapcsolatot, végül regisztrálja a végpontokat és a WebSocket-kezelőt ugyanazon a HTTP-szerveren, amely a 3000-es porton figyel. A feltöltött fájl legnagyobb mérete 10 MB, egy WebSocket-üzeneté 1 KB.

API-végpontok

12. táblázat: A REST API végpontjai

Végpont	Ki éri el	Leírás
GET /api/me	mindenki	Ki van bejelentkezve

Végpont	Ki éri el	Leírás
POST /register, /login, /logout	mindenki	Regisztráció, belépés, kilépés
GET /api/subjects	mindenki	Tantárgyak listája
GET /api/tutors	mindenki	Oktatók listája szűréssel
GET /api/tutors/:id/reviews	mindenki	Egy oktató értékelései
GET /api/tutors/:id/availability	mindenki	Egy oktató szabad időpontjai
GET /api/stats	mindenki	Főoldali statisztikák
GET, POST /api/conversations	bejelentkezett	Beszélgetések, új beszélgetés
GET, POST /api/conversations/:id/messages	résztvevő	Üzenetek lekérése, küldése
POST /api/conversations/:id/attachments	résztvevő	Üzenet csatolmánnyal (max. 10 MB)
POST /api/conversations/:id/read	résztvevő	Üzenetek olvasottnak jelölése
GET /api/messages/:id/attachment	résztvevő	Csatolmány megnyitása
PATCH, DELETE /api/messages/:id	küldő	Üzenet szerkesztése, visszavonása
GET, PUT, DELETE /api/account	bejelentkezett	Saját fiók lekérése, módosítása, törlése
GET, POST /api/availability	oktató	Szabad időpontok megadása
DELETE /api/availability/:id	oktató	Szabad időpont törlése
POST /api/bookings	diák	Foglalási kérelem
GET /api/bookings, /summary	bejelentkezett	Saját foglalások
POST /api/bookings/:id/confirm, /reject	oktató	Foglalás elfogadása, elutasítása
POST /api/bookings/:id/cancel	diák, oktató	Foglalás lemondása
POST /api/bookings/:id/review	diák	Értékelés leadása
GET /api/students/:id/profile	oktató	A beszélgetőpartner diák profilja

Hitelesítés és jogosultságkezelés

Bejelentkezéskor a szerver először ellenőrzi, hogy nincs-e üres mező, majd az e-mail-cím alapján megkeresi a felhasználót, és az Argon2 könyvtár összeveti a megadott jelszót a tárolt hash-sel. Sikeres ellenőrzés után a szerver a munkamenetben eltárolja a felhasználó azonosítóját, nevét, e-mail-címét és szerepkörét; a böngésző csak a munkamenet-azonosítót kapja meg sütiben. Regisztrációkor a szerver ellenőrzi, hogy minden mező ki van-e töltve, egyezik-e a két jelszó, a név legfeljebb 60, az e-mail-cím legfeljebb 100 karakter-e, a szerepkör csak diák vagy korrepetáló lehet, az e-mail-cím pedig egyedi.

Minden védett kérésnél a szerver megvizsgálja a munkamenetet: ha a felhasználó nincs bejelentkezve, 401-es, ha nincs jogosultsága a művelethez, 403-as kóddal válaszol. Másik felhasználó beszélgetésének vagy foglalásának elérésekor 404-es vagy 403-as válasz érkezik, törölt vagy hiányzó munkamenet esetén a WebSocket-kapcsolat megszakad. A feltöltött fájlok véletlenszerű nevet kapnak, csak engedélyezett kiterjesztéssel (kép, PDF,

Word, Excel, PowerPoint, szöveg) menthetők, és csak a beszélgetés két tagja nyithatja meg őket.

Hibakezelés

13. táblázat: Hibakezelés státuszkódok szerint

Státuszkód	Mikor	Mit lát a felhasználó
400	Üres vagy hibás mező, túl hosszú szöveg, nem engedélyezett fájl	Magyar nyelvű hibaüzenet, pl. „Minden mezőt ki kell tölteni.”
401	Nincs bejelentkezve	„Nem vagy bejelentkezve.”
403	Nincs joga a művelethez	Pl. „Csak diák foglalhat időpontot.”
404	A keresett elem nem létezik	Pl. „Az időpont nem található.”
409	Az állapot időközben megváltozott	Pl. a foglalás állapota már módosult.
500	Adatbázishiba	Általános üzenet; a részleteket a szerver naplózza.

Az adatbázishibákat a szerver a konzolra írja, a felhasználónak azonban csak rövid üzenetet küld, és hiba után is tovább fut. Kliensoldalon a `fetch` hívások hibáit az oldal elkapja, és az `error` mező szövegét megjeleníti a felhasználónak.

4.7 Kliensoldali megvalósítás (frontend)

A kliensoldali fájlok a `vsc/` mappában találhatók: `html/` (oldalak), `css/` (stílusok, közös `style.css`), `js/` (oldalankénti szkriptek és a `creator.js`) és `images/` (képek). A `creator.js` minden oldalon fut. A böngésző ellenőrzi a kötelező mezők kitöltését és a két jelszó egyezését, a jelszó mutatható vagy elrejthető, a hibákat pedig az oldal azonnal kiírja.

14. táblázat: Oldalak, szkriptek és API-hívások

Oldal	Szkriptek	Hívott végpontok
index.html (főoldal)	creator.js, home.js	/api/stats
subs.html (tantárgyak)	creator.js, dropdown.js, index.js	/api/subjects, /api/tutors
oktatok.html	creator.js, oktatok.js, selectmenu.js, booking.js	/api/subjects, /api/tutors, /api/bookings
chat.html	creator.js, chat.js, booking.js	/api/conversations, /api/messages, /api/students
foglalas.html	creator.js, foglalas.js	/api/availability, /api/bookings
foglalasutan.html	creator.js, foglalasutan.js	/api/bookings, /api/bookings/summary
login, register, gyik	creator.js	/api/me, /api/account, /logout

A kliens a `fetch` függvénnyel hívja az API-t. A `credentials: "include"` beállítás miatt a böngésző a munkamenet-sütit is elküldi, így a szerver felismeri a felhasználót:

```
const response = await fetch("/api/me", {
    method: "GET",
    credentials: "include"
});
const data = await response.json();
```

4.8 Telepítés és üzemeltetés

A futtatáshoz Node.js és npm, XAMPP (MariaDB 10.4 vagy újabb), egy modern webböngésző és Git szükséges; a Visual Studio Code használata nem kötelező. Az asztali alkalmazáshoz Visual Studio és .NET 10 kell, a weboldalhoz erre nincs szükség. A 3000-es portnak szabadnak kell lennie.

A telepítés lépései: (1) a tároló klónozása vagy ZIP-ben való letöltése; (2) a csomagok telepítése Windowson az `installer.bat` futtatásával, vagy az `npm install` paranccsal; (3) az adatbázis létrehozása. Ehhez a XAMPP-ban el kell indítani a MySQL szolgáltatást, a phpMyAdminban `zsamo` néven `utf8mb4_hungarian_ci` egybevetéssel létre kell hozni az adatbázist, majd importálni kell az `sql/zsamo.sql` fájlt. Sikeres importálás után 9 tábla jelenik meg.

```
git clone https://github.com/Matyi24/-TutorConnect-.git
cd -TutorConnect-
npm install
mysql -u root -e "CREATE DATABASE zsamo"
mysql -u root zsamo < sql/zsamo.sql
npm run dev        # fejlesztői indítás, módosításkor újraindul
```

15. táblázat: Konfigurációs beállítások

Beállítás	Helye	Alapérték
Adatbázis-kapcsolat	vsc/js/main.js	localhost, root, jelszó nélkül, zsamo
Munkamenet titkos kulcsa	SESSION_SECRET	fejlesztői alapérték
Tesztnaplózás	NODE_ENV, DEBUG_LOGS	fejlesztéskor bekapcsolva
Port	main.js	3000
Süti secure jelzője	main.js	false (HTTP-hez)

Éles használat előtt be kell állítani a `SESSION_SECRET` változót és a `NODE_ENV=production` módot, mert fejlesztői módban a naplózás érzékeny adatokat is kiírhat a konzolra. HTTPS használata esetén a munkamenet-süti `secure` beállítását `true` értékre kell állítani. A bejelentkezések a szerver memóriájában tárolódnak, ezért a szerver újraindítása után mindenkinek újra be kell lépnie.

4.9 Tesztelés

A tesztelést három módszerrel végeztük:

- **Automatizált tesztek:** a szerver végpontjait és a jelszókezelést a Node.js beépített tesztfuttatója ellenőrzi (OpenJS Foundation, 2026a). A tesztekhez nem kell valódi MySQL-szerver: a `mysql2` modult egy memóriában működő, hamis adatbázis helyettesíti, a valódi szerver pedig véletlen porton indul el.
- **Kézi tesztelés:** böngészőben végigkattintottuk az oldalakat vendégként, diákként és korrepetálóként is. Ez volt a leggyakrabban használt módszer.
- **Reszponzivitási próba:** az oldal megjelenését telefonon és közepes méretű monitoron ellenőriztük.

16. táblázat: Fontosabb tesztesetek

Teszteset	Várt eredmény
Regisztráció üres mezővel	400, hibaüzenet
Regisztráció eltérő jelszavakkal	400, „A két jelszó nem egyezik.”
Bejelentkezés rossz jelszóval	Sikertelen belépés
Védett végpont bejelentkezés nélkül	401
Diák szabad időpontot hirdet	403
Idegen beszélgetés olvasása	Elutasítás
2000 karakternél hosszabb üzenet	Elutasítás
Nem engedélyezett fájl feltöltése	Elutasítás
Csatolmány elérése másik felhasználóval	404
Értékelés az 1–5 csillagos tartományon kívül	400
Üzenet szerkesztése és visszavonása	Csak a küldő teheti meg
Fiók törlése	Az adatok anonimizálódnak, a felhasználó kijelentkezik
60 karakternél hosszabb név / 100 karakternél hosszabb e-mail	400
Regisztráció admin szerepkörrel	Elutasítás, csak diák vagy korrepetáló lehet
Túl hosszú értékelés szöveg	400

Az automatizált tesztkészlet 2 fájlban (`auth.test.js`, `main.test.js`) 22 csoportba rendezve 141 tesztet tartalmaz, amelyek lefedik a jelszókezelést, az oldalak kiszolgálását, a belépést, az adatlekérdezéseket, a chatet, a fájlkezelést, a WebSocket-értesítéseket és a fiókkezelést.

17. táblázat: Az utolsó tesztfuttatás eredménye

Összes teszt	Sikeres	Sikertelen	Megszakadt
141	116 (82,3%)	13 (9,2%)	8 (5,7%)

A három kategória összege 137; a fennmaradó 4 teszt nem kapott eredményt. Két tesztet szándékosan TODO-ként jelöltünk meg: a feltöltött fájlok közvetlen elérését és a túl nagy

WebSocket-üzenet hibakezelését. A sikertelen tesztek főként a chatcsatolmányok fel- és letöltését, az üzenetküldés és -visszavonás néhány esetét, valamint a fiókadatok lekérését és módosítását érintik.

Feltárt hibák és javításuk

18. táblázat: Feltárt hibák és javításuk

Hiba	Javítás
A szerver forráskódja elérhető volt a /js útvonalon	Fehérlistás kiszolgálás: csak a kliensfájlok érhetők el.
A chatfájlok közvetlenül elérhetők voltak	Véletlen fájlnév; csak a beszélgetés tagjai tölthetik le.
Az oktatók e-mail-címe kiszivároghatott	Az e-mail-cím nem kerül bele az API-válaszokba.
Az ékezetes fájlnevek (ő, ű) elromlottak	A név újrakódolása UTF-8-ra, a veszélyes karakterek törlése.
A lejárt foglalások beragadtak	Lekérés előtt a lejártak automatikusan COMPLETED vagy CANCELLED állapotba kerülnek.
Lemondás után az időpont foglalt maradt	Elutasításkor és lemondáskor az időpont újra szabaddá válik.
A törölt fiók munkamenete tovább élt	A szerver kijelentkezteti a felhasználót, a WebSocket-kapcsolat megszakad.
Az e-mail mező túl rövid volt	Az adatbázismező szükség esetén 100 karakteresre bővül.
A régi feltöltések a js mappában voltak	A szerver áthelyezi őket a feltöltések mappájába.
A sötét mód és a tantárgyak oldal kinézete hibás volt	Újratervezés, egyedi legördülő menük.
Túl nagy WebSocket-üzenet leállíthatja a szervert	Még nincs javítva (a tesztekben TODO).

4.10 Forráskód és projektstruktúra

A forráskód a GitHubon érhető el (https://github.com/Matyi24/-TutorConnect-). A tároló 2026. szeptember 1-jén jött létre; október 13-ig több mint 150 commit készült, amelyeket a fő (`main`) ágban egyesítettünk, a JavaScript-kód mérete körülbelül 12 600 sor. A legnagyobb fájlok a `main.js` (kb. 3850 sor), a `chat.js` (kb. 2800 sor), a `main.test.js` (kb. 1930 sor), a `creator.js` (kb. 1770 sor) és az `oktatok.js` (kb. 1440 sor).

```
-TutorConnect-/
  installer.bat          csomagok telepítése
  package.json           csomagok és szkriptek
  sql/zsamo.sql          adatbázis + mintaadatok
  vsc/                   a weboldal
    html/  css/  js/  images/
    test/                automatizált tesztek
  Desktop/TutorConnectApp/   asztali alkalmazás (C#)
  dokumentáció/
```

4.11 Csapatmunka

A csapat három tagja a saját gépén fejlesztett, a változásokat GitHubra töltötte fel, és a fő ágban egyesítette őket. A feltöltött felhasználói fájlokat a `.gitignore` kizárja a verziókezelésből. A közös teendőlistát a kódban (`vsc/js/npm_list`) vezettük.

19. táblázat: A csapattagok hozzájárulása

Csapattag	Fő feladatai	Technológiák
Porkoláb Mátyás Attila	A főoldal első változata, navigációs sáv és lábléc (creator.js); belépés és regisztráció, jelszó hash-elés (auth.js) és szerveroldali ellenőrzések; profil és fiókbeállítások; a sötét mód első része; automatizált tesztek, telepítő és dokumentáció.	JavaScript, HTML, CSS, Node.js, Argon2id
Stickel Zsombor	A főoldal kinézete, a tantárgyak oldal kártyákkal és kategóriaváltóval, animációk; az adatbázis első terve és tesztadatok; valós idejű chat fájl- és képküldéssel, üzenetszerkesztéssel; a foglalás és az értékelés kidolgozása; biztonsági javítások; sötét mód minden oldalra.	Node.js, Express, WebSocket, MySQL, JavaScript, CSS
Marián Máté	A projekt alapjai (package.json, nodemon), a szerver első változata és az oldalak közötti váltás; az adatbázis feltöltése; az oktatók oldal az értékelésekkel; GYIK oldal; a foglalás oldal első változata; a C# asztali alkalmazás.	JavaScript, Node.js, MySQL, HTML, CSS, C#

A projekt során mindenki segítette a többieket, így minden csapattag legalább kis mértékben minden területen dolgozott. 2026. szeptember 1. és október 13. között Porkoláb Mátyás Attila 48, Stickel Zsombor 72, Marián Máté 34 commitot készített.

20. táblázat: A fejlesztés időrendje

Időszak	Elkészült részek
Szept. 1–4.	A főoldal első változata, a tantárgyak oldal, az adatbázis első terve.
Szept. 8–18.	Kártyák és design, GYIK, belépés és regisztráció, jelszó hash-elés, a chat sablonja.
Szept. 22–30.	Az adatbázis bővítése, az oktatók oldal, a foglalás első változata, a profil oldal.
Okt. 1–9.	Valós idejű chat, profilszerkesztés, foglalás és értékelés, biztonság, tesztek.
Okt. 11–13.	Sötét mód minden oldalra, az oktatók és a tantárgyak oldal újratervezése.

5. Következtetések, javaslatok, összegzés

A TutorConnect célja az volt, hogy a diákok és a korrepetálók gyorsan egymásra találjanak, egyeztessenek és időpontot foglaljanak. Ez a cél megvalósult: a kész rendszerben lehet regisztrálni, tantárgy és óradíj szerint oktatót keresni, valós időben csevegni fájlküldéssel, időpontot foglalni az oktató általi jóváhagyással, és az óra után értékelést adni. A rendszer 9 adatbázistáblára, több mint 30 API-végpontra és 4 szerepkörre épül, működését 141 automatizált teszt ellenőrzi.

5.1 Tapasztalatok

- **Csapatmunka és verziókezelés:** a Git és a GitHub nélkül három fejlesztő párhuzamos munkája nem lett volna összehangolható; a gyakori, jól leírt commitok sokat segítettek.
- **Biztonság:** a fejlesztés során több biztonsági hibát is feltártunk és javítottunk (elérhető forráskód, védtelen feltöltések, kiszivárgó e-mail-címek). Megtanultuk, hogy a jogosultságkezelést minden végponton szerveroldalon kell ellenőrizni.
- **Tesztelés:** az automatizált tesztek sok hibát már a fejlesztés közben megfogtak; a hamis adatbázissal futó tesztkörnyezet gyorssá és megismételhetővé tette a futtatást.
- **Mesterséges intelligencia:** az MI-alapú asszisztens (Anthropic, 2026) hasznos volt tanácsadásra és kódellenőrzésre, de a javaslatait minden esetben ellenőrizni és tesztelni kellett.

5.2 Továbbfejlesztési javaslatok

- tartós munkamenet-tár bevezetése, hogy a szerver újraindítása után is bejelentkezve maradjanak a felhasználók;
- HTTPS használata, valamint az adatbázis-beállítások áthelyezése környezeti változókba;
- a túl nagy WebSocket-üzenetek szabályos kezelése és a sikertelen tesztek javítása;
- online fizetés az órákhoz;
- adminisztrációs felület kialakítása és az asztali alkalmazás befejezése;
- a megjelenés tesztelése további eszközökön (régebbi monitorok, táblagépek).

5.3 Összegzés

Pályamunkánkban egy működő, háromrétegű webalkalmazást készítettünk, amely a korrepetálás teljes folyamatát – a kereséstől az értékelésig – egyetlen felületen támogatja. A projekt során a gyakorlatban alkalmaztuk a Node.js és Express alapú szerverfejlesztést, a relációs adatbázis-tervezést, a valós idejű kommunikációt és a biztonságos jelszókezelést, miközben csapatként, verziókezelő rendszerrel dolgoztunk. Úgy gondoljuk, hogy a TutorConnect a továbbfejlesztési javaslatok megvalósításával éles környezetben is használható szolgáltatássá válhat.

6. Irodalomjegyzék

Althoff, R. (2026). *node-argon2* (Version 0.45.1) [Computer software]. npm. https://www.npmjs.com/package/argon2

Anthropic. (2026). *Claude* [Large language model]. https://claude.ai

Apache Friends. (2026). *XAMPP* [Computer software]. https://www.apachefriends.org

Biryukov, A., Dinu, D., Khovratovich, D., & Josefsson, S. (2021). *Argon2 memory-hard function for password hashing and proof-of-work applications* (RFC 9106). Internet Engineering Task Force. https://doi.org/10.17487/RFC9106

Express.js. (2026a). *express-session* (Version 1.19.0) [Computer software]. npm. https://www.npmjs.com/package/express-session

Express.js. (2026b). *multer* (Version 2.4.0) [Computer software]. npm. https://www.npmjs.com/package/multer

Fette, I., & Melnikov, A. (2011). *The WebSocket protocol* (RFC 6455). Internet Engineering Task Force. https://doi.org/10.17487/RFC6455

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

Git Project. (2026). *Git* [Computer software]. https://git-scm.com

GitHub. (2026). *GitHub* [Online software development platform]. https://github.com

MariaDB Foundation. (2026). *MariaDB documentation*. https://mariadb.com/docs

Microsoft. (2026a). *Visual Studio Code* [Computer software]. https://code.visualstudio.com

Microsoft. (2026b). *Visual Studio* [Computer software]. https://visualstudio.microsoft.com

Microsoft. (2026c). *.NET* (Version 10) [Computer software]. https://dotnet.microsoft.com

Mozilla. (2026). *MDN Web Docs*. https://developer.mozilla.org

OpenJS Foundation. (2026a). *Node.js* [Computer software]. https://nodejs.org

OpenJS Foundation. (2026b). *Express* (Version 5.2.1) [Computer software]. https://expressjs.com

OWASP Foundation. (2026). *Password storage cheat sheet*. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

phpMyAdmin Project. (2026). *phpMyAdmin* [Computer software]. https://www.phpmyadmin.net

Pinca, L. (2026). *ws: A Node.js WebSocket library* (Version 8.22.0) [Computer software]. npm. https://www.npmjs.com/package/ws

Sidorov, A. (2026). *mysql2* (Version 3.24.5) [Computer software]. npm. https://www.npmjs.com/package/mysql2